Naukri.com Automation & Profile Booster

Complete Technical Architecture, System Design & Operations Guide

Java 17+ | Playwright 1.49.0 | Jackson 2.18.2 | Daily 8:30 AM IST | Enterprise Ready

1. Executive Summary

The **Naukri Automation Bot** is an enterprise-grade background automation engine engineered to maximize candidate visibility and streamline the job application process on **Naukri.com**.

Core Capabilities:

- **Profile Freshness Optimization:** Recruiter search algorithms prioritize recently updated candidate profiles. The bot automatically touches the resume headline daily, tagging the account as "*Updated Today*" to increase recruiter impressions by 3–5x.
- **Automated Targeted Applications:** Automatically searches for postings matching configured keywords, experience levels, and locations (including Remote), executing 1-Click *Easy Apply* while bypassing dead-end third-party redirects.
- **Deduplication Database:** Persists applied job IDs and metadata in a structured JSON database to ensure no job is ever applied to more than once.
- **Anti-Detection & Human Mimicking:** Utilizes Playwright Chromium stealth modifications, randomized typing cadences, and realistic mouse movements to safeguard candidate accounts.

2. Technology Stack & Architectural Components

Component	Technology	Version	Key Responsibility
Core Language	Java (JDK)	17 / 18 / 21 LTS	Strong-typed, thread-safe enterprise runtime.
Browser Automation	Microsoft Playwright	1.49.0	Headless / Visible Chromium control & network interception.
JSON & YAML Binding	Jackson Databind / YAML	2.18.2	Parses `config.yaml` and tracks `applied_jobs.json`.
Logging Engine	SLF4J + Logback Classic	1.5.16	Colorized terminal output and daily rolling log files.
Build & Packaging	Maven Shade Plugin	3.5.3	Builds a self-contained executable Fat JAR (`naukri-automation.jar`).

3. Project Directory Architecture

```
naukri-automation/
├── pom.xml                              # Maven dependencies & shaded fat JAR build
├── .env.example                         # Template for user credentials and runtime flags
├── .env                                 # Local environment file (Ignored in Git)
├── config.yaml                          # Job search keywords, filters, and criteria
├── run_daily.bat                        # Windows batch execution script
├── setup_windows_task.ps1               # Windows Task Scheduler auto-registration
├── README.md                            # Quick-start documentation
├── DOCUMENTATION.md                     # Full technical documentation
├── .github/
│   └── workflows/
│       └── daily_naukri.yml            # Cloud GitHub Actions workflow (08:30 AM IST)
├── data/
│   ├── session_state.json               # Saved cookies & local storage
│   └── applied_jobs.json                # Deduplication database of applied job IDs
├── logs/
│   └── naukri_automation.log            # Daily rolling execution logs
└── src/
    └── main/
        ├── resources/
        │   └── logback.xml              # Log pattern and log rotation policy
        └── java/com/naukri/automation/
            ├── Main.java                # Entrypoint & CLI execution coordinator
            ├── Scheduler.java           # Daily 08:30 AM IST cron scheduler
            ├── config/
            │   ├── AppConfig.java       # Strong-typed Java POJO configuration model
            │   └── ConfigLoader.java    # YAML and .env parser
            ├── browser/
            │   └── BrowserManager.java  # Playwright Chromium manager with stealth flags
            ├── service/
            │   ├── AuthService.java     # Login and OTP detection handler
            │   ├── ProfileService.java  # Profile headline touch optimizer
            │   ├── JobSearchService.java # Search query generator & DOM scraper
            │   ├── ApplierService.java  # Easy-apply clicker & modal questionnaire handler
            │   └── TrackerService.java  # JSON history tracker and counter
            └── model/
                ├── JobListing.java      # Model representing a discovered job
                ├── AppliedJobInfo.java  # Model representing an applied job history record
                └── AutomationStats.java # Metrics model (Scanned, Applied, Skipped)
```

4. Core Services & Implementation Details

4.1 BrowserManager (Browser Lifecycle & Anti-Detection)

Located at `com.naukri.automation.browser.BrowserManager`. Configures Chromium with anti-bot evasion arguments:

- `--disable-blink-features=AutomationControlled`: Removes automated browser flag.

- Injects `navigator.webdriver = undefined` into the context before script evaluation.
- Saves browser session state (cookies, local storage) into `data/session_state.json` for seamless future logins.

4.2 AuthService (Authentication & OTP Handling)

Located at `com.naukri.automation.service.AuthService`. Checks session status via `/mnjuser/profile`. If expired, authenticates at `/nlogin/login` using randomized human keystroke pacing (60ms). If an OTP challenge is triggered, pauses for 60s in visible mode (`HEADLESS=false`) to allow the user to complete verification once.

4.3 ProfileService (Profile Booster)

Located at `com.naukri.automation.service.ProfileService`. Navigates to the user's Naukri profile, edits the Resume Headline textarea, toggles a trailing period (`.`), and clicks Save. This simple operation refreshes the timestamp on Naukri's indexer, placing the candidate profile in the top tier of recruiter candidate searches.

4.4 JobSearchService & ApplierService

`JobSearchService` builds structured query URLs and extracts job cards via DOM locators. `ApplierService` verifies that the job does not belong to an excluded company or title keyword, checks deduplication history, clicks *Apply*, and dismisses questionnaire dialogues.

5. Configuration Reference

5.1 Environment Variables (`.env`)

```
# User Credentials
NAUKRI_EMAIL=your_email@example.com
NAUKRI_PASSWORD=your_naukri_password

# Execution Settings
HEADLESS=false                    # Set false on 1st run to solve OTP if prompted
UPDATE_PROFILE=true               # Boosts resume headline daily
MAX_APPLICATIONS_PER_RUN=25       # Daily application cap

# Scheduler Settings
SCHEDULE_TIME=08:30               # 24-hour format
TIMEZONE=Asia/Kolkata
```

5.2 Search & Filters (`config.yaml`)

```
search_criteria:
  keywords:
    - "Full Stack Developer"
    - "React Developer"
    - "Java Developer"
  locations:
    - "Bengaluru"
    - "Hyderabad"
    - "Remote"
  experience:
    min: 2
    max: 5
  freshness_days: 1 # Jobs posted in last 24 hours

filters:
  exclude_companies:
    - "Confidential"
    - "Staffing"
  exclude_title_keywords:
    - "Intern"
    - "Trainee"
    - "Tech Lead"
  easy_apply_only: true
```

6. Execution & Deployment Strategies

Option 1: Windows Task Scheduler (Recommended for Local PC)

> Run PowerShell as Administrator and execute:
> `powershell -ExecutionPolicy Bypass -File setup_windows_task.ps1`
>
> This registers a task named **NaukriDailyJobApply** that triggers `run_daily.bat` automatically every morning at **8:30 AM IST**.

Option 2: Cloud Automation via GitHub Actions (100% Free Serverless)

> 1. Push this repository to a **Private** GitHub repository.
> 2. In GitHub, go to **Settings** > **Secrets and variables** > **Actions**.
> 3. Add `NAUKRI_EMAIL` and `NAUKRI_PASSWORD`.
> 4. The pre-configured workflow at `.github/workflows/daily_naukri.yml` runs every morning at **8:30 AM IST (03:00 AM UTC)**.

Option 3: Manual CLI Execution

```
# Run once immediately
java -jar target/naukri-automation.jar

# Run immediate test + start daily 8:30 AM scheduler daemon
java -jar target/naukri-automation.jar --run-now
```